UniFECAF

Object Oriented Programming

Sistema Automotivo
Gestão de Estoque de Veículos

Relatório teórico — Parte 1

Trabalho acadêmico de programação orientada a objetos.
Java 21 · Spring Boot 4.1.2 · MySQL 8.4

31 de outubro de 2026

1. Problema e objetivo

Concessionárias que utilizam anotações manuais ou sistemas fragmentados podem manter informações desatualizadas sobre os veículos disponíveis. A falta de organização dificulta encontrar modelos, consultar preços e atualizar o estoque. Vendedores perdem tempo no atendimento, clientes recebem informações pouco claras e administradores têm menor visibilidade do inventário.

O objetivo do sistema é centralizar as informações de veículos e disponibilizar cadastro, consulta com filtros, atualização e remoção, preservando os dados em MySQL. A solução foi desenvolvida como API REST em Java com Spring Boot. O frontend opcional foi implementado em Java com Vaadin Flow, com telas de veículos, marcas e modelos.

As principais barreiras são a familiaridade com Java e Spring, a modelagem de relacionamentos no banco e a integração das camadas. A continuidade do problema tende a manter erros de gestão e perda de produtividade. Relatórios financeiros, autenticação e processamento de vendas não fazem parte do escopo solicitado.

2. Solução e tecnologias

Tecnologia	Justificativa e consideração
Java 21	Representação do domínio em classes e aplicação de encapsulamento, métodos e herança. Exige compreender os fundamentos da linguagem.
Spring Boot 4.1.2	Configuração da aplicação, API REST com Spring Web MVC, validação de entradas e integração com persistência. Exige compreender o fluxo entre camadas.
MySQL 8.4	Persistência relacional de marcas, modelos e veículos, com chaves estrangeiras e restrições. Exige servidor configurado e conhecimento de SQL.
Spring Data JPA / Hibernate	Mapeamento entre objetos e tabelas e execução das operações de persistência.
Cliente HTTP	Permite testar e demonstrar a API. Complementa a interface Vaadin nas verificações da API.

3. Levantamento de requisitos

Entrevista **simulada** com um responsável pelo estoque de uma concessionária. As respostas abaixo representam o cenário adotado e não uma entrevista realizada com uma pessoa real.

1. Qual problema o sistema deve resolver?

Resposta simulada: Organizar o estoque e permitir consulta dos dados sem depender de anotações espalhadas.

2. Quem utilizará a solução?

Resposta simulada: Vendedores e administradores responsáveis pelo estoque.

3. Quais dados são necessários em um veículo?

Resposta simulada: Marca, modelo, ano de fabricação, cor, preço, quilometragem e status; tipo também será registrado para o filtro citado no desafio.

4. Como marcas e modelos se relacionam?

Resposta simulada: Cada modelo pertence a uma marca, e cada veículo se associa a um modelo cadastrado.

5. Quais status serão utilizados?

Resposta simulada: Disponível, vendido e descontinuado.

6. Como os veículos serão encontrados?

Resposta simulada: Por marca, modelo, preço, ano, status e tipo.

7. Os filtros podem ser combinados?

Resposta simulada: Sim. Todos os critérios informados devem ser atendidos simultaneamente.

8. Quais informações precisam ser atualizadas?

Resposta simulada: Preço, quilometragem e status de disponibilidade.

9. O que acontece quando um veículo é vendido?

Resposta simulada: Ele pode ser marcado como vendido e posteriormente removido do estoque por exclusão explícita.

10. Quais veículos podem ser removidos?

Resposta simulada: Os vendidos ou descontinuados; para um disponível, deve-se primeiro atualizar o status.

11. Quais valores de preço e quilometragem serão aceitos?

Resposta simulada: Preço positivo com até duas casas decimais; quilometragem inteira e não negativa.

12. Os dados precisam permanecer após reiniciar?

Resposta simulada: Sim. Os registros devem ser persistidos no MySQL.

13. Como evitar cadastro duplicado de marca/modelo?

Resposta simulada: A marca tem nome único; um modelo não pode repetir o nome dentro da mesma marca.

14. O que caracteriza facilidade de uso?

Resposta simulada: Operações com nomes claros, exemplos de requisições e mensagens compreensíveis para entradas inválidas.

15. Como verificar rapidez nas consultas e atualizações?

Resposta simulada: Executar consultas, filtros e atualizações em ambiente local e registrar os tempos observados.

4. Requisitos funcionais

ID	Requisito definido
RF01	Cadastrar veículo com modelo, marca associada, ano de fabricação, cor, preço, quilometragem, status e tipo.
RF02	Cadastrar e listar marcas.
RF03	Cadastrar e listar modelos vinculados às marcas.
RF04	Listar veículos e consultar veículo pelo identificador.
RF05	Filtrar veículos por marca, modelo, faixa de preço, ano e status; incluir tipo para cobrir o desafio inicial.
RF06	Atualizar preço, quilometragem e status.
RF07	Remover veículos vendidos ou descontinuados.

5. Requisitos não funcionais

Requisito	Tratamento na implementação
Usabilidade	Rotas e campos consistentes, coleção de requisições, README com sequência de cadastro e mensagens de validação. A interface Vaadin apresenta formulários, seleção de marcas/modelos por nome, filtros, mensagens de validação e confirmação de exclusão.
Performance	Filtros executados no banco; índices em preço, ano e status; associações carregadas com EntityGraph nas consultas de veículos; tempos locais registrados no PDF prático.
Persistência	MySQL armazena os registros. Reiniciar a API preserva os dados; o teste manual está documentado no PDF prático.

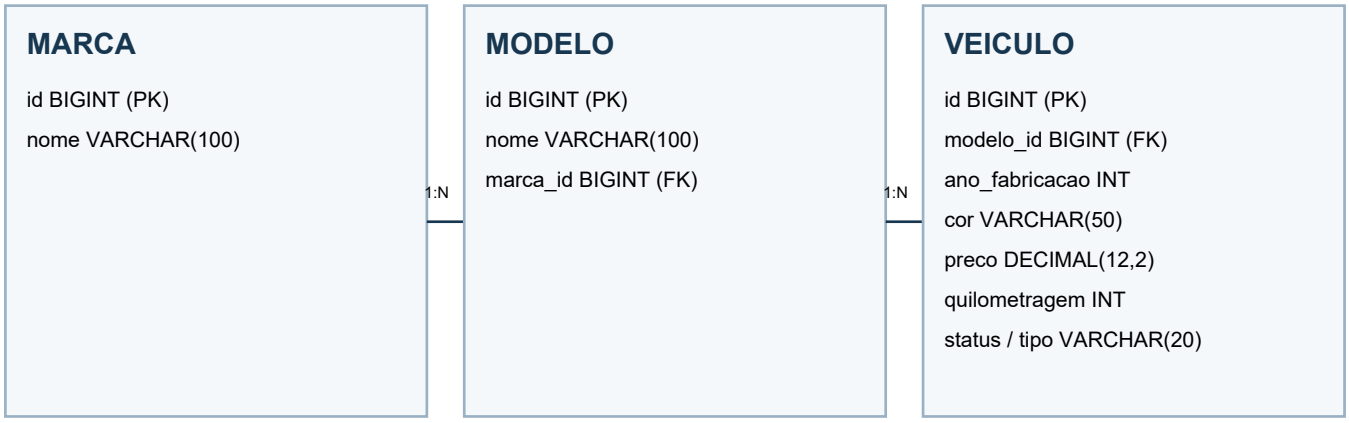
6. Regras de negócio

O modelo deve existir antes do veículo, e a marca deve existir antes do modelo. A marca do veículo é derivada do modelo, evitando combinações incompatíveis. Preço deve ser positivo e caber em DECIMAL(12,2); quilometragem não pode ser negativa. Como regra de consistência do projeto, o ano de fabricação aceito vai de 1886 ao ano atual.

Status permitidos: DISPONIVEL, VENDIDO e DESCONTINUADO. Tipos permitidos: CARRO, MOTO e UTILITARIO. Nomes de marca não se repetem; nomes de modelo não se repetem dentro da mesma marca. A remoção é permitida somente para vendido ou descontinuado e não apaga marca ou modelo. Essas regras detalham o cenário simulado; o PDF não fixa todos esses valores.

7. Entidades e relacionamentos

Classe	Atributos e responsabilidade
EntidadeBase	id: Long. Superclasse abstrata com identificação reutilizada pelas entidades.
Marca	id herdado; nome: String. Representa a fabricante.
Modelo	id herdado; nome: String; marca: Marca. Representa um modelo de uma fabricante.
Veiculo	id herdado; modelo: Modelo; anoFabricacao: Integer; cor: String; preco: BigDecimal; quilometragem: Integer; status: StatusVeiculo; tipo: TipoVeiculo. Representa uma unidade em estoque.
Enums	StatusVeiculo e TipoVeiculo limitam os valores aceitos. Não são tabelas separadas.



Uma marca possui zero ou muitos modelos. Cada modelo possui uma marca. Um modelo pode identificar zero ou muitos veículos; cada veículo possui um modelo. O acesso à marca é feito por veiculo.modelo.marca. No modelo físico, modelo.marca_id e veiculo.modelo_id são chaves estrangeiras.

8. Aplicação de orientação a objetos

Classes e objetos: Marca, Modelo e Veiculo definem estruturas do domínio. Um Corolla prata de 2022 cadastrado é um objeto Veiculo específico, com ID próprio; outro veículo do mesmo modelo pode ter preço, cor e quilometragem diferentes.

Atributos: os campos privados armazenam o estado. BigDecimal foi utilizado para preço e mapeado para DECIMAL(12,2). As associações são referências a objetos Modelo e Marca, e não textos independentes sem vínculo.

Métodos: o construtor de Veiculo organiza o cadastro e valida ano, cor e associações. atualizarDadosComerciais altera preço, quilometragem e status; podeSerRemovido informa se o status permite exclusão.

Encapsulamento: os atributos são privados e não há setters públicos indiscriminados. A atualização passa por um método que verifica os três valores antes de mudar o estado, impedindo preço inválido, quilometragem negativa e status ausente.

Herança: Marca, Modelo e Veiculo estendem EntidadeBase e herdam o id com geração automática. A anotação @MappedSuperclass reutiliza o mapeamento nas três tabelas, sem criar uma tabela de superclasse. Trata-se de herança de implementação; a relação Veiculo–Modelo é associação, não herança.

9. Arquitetura e conclusão

O controller recebe HTTP e valida DTOs. O service coordena as regras e as transações. O repository utiliza Spring Data JPA e o MySQL para persistência. As respostas são DTOs JSON; exceções conhecidas são convertidas em códigos HTTP e mensagens compreensíveis. Os filtros são combinados com lógica E e executados no banco.

A implementação contempla o CRUD e as relações de estoque definidos no trabalho. O código e os testes estão no repositório público, e as capturas de execução e os códigos principais constam no relatório prático. A interface Vaadin permite realizar essas operações pelo navegador e utiliza as mesmas regras dos serviços da API.

10. Referências

UniFECAF. TRABALHO — Object Oriented Programming.docx.pdf. Enunciado da atividade, 6 páginas.

Spring. Accessing Data with JPA. Documentação oficial. Acesso em: 31 out. 2026.

Spring. Validation — Spring MVC. Documentação oficial. Acesso em: 31 out. 2026.

Spring. SQL Databases — Spring Boot. Documentação oficial. Acesso em: 31 out. 2026.

MySQL. CREATE TABLE — Reference Manual. Documentação oficial. Acesso em: 31 out. 2026.

Código-fonte: https://github.com/igorguimaraes-dev/sistema-automotivo-poo.

Interface em Java com Vaadin

Vaadin Flow 25.3.4 permite construir as telas com componentes Java. As classes ficam em view; a aparência é definida por CSS. VeiculosView apresenta tabela, filtros e operações de estoque. MarcasView e ModelosView apresentam os cadastros relacionados. VeiculoDialog valida os DTOs e chama os services, preservando as regras de negócio e a precisão de BigDecimal. A API REST continua disponível para outros clientes.

Vaadin. Documentação oficial do Vaadin Flow. Acesso em: 31 out. 2026.